

LLaMa

introduction

Large Languages Models (LLMs) trained on massive corpora of texts have shown their ability to perform new tasks from textual instructions or from a few examples (Brown et al., 2020).

대규모 텍스트 코퍼스로 학습된 LLMs는 textual instructions나 few texamples에서 새로운 작업을 수행할 수 있는 능력을 보여줬다. (GPT-3에 대한 이야기)

These few-shot properties first appeared when scaling models to a sufficient size (Kaplan et al., 2020), resulting in a line of work that focuses on further scaling these models (Chowdhery et al., 2022; Rae et al., 2021).

이런 few-shot 속성은 모델을 충분한 크기로 scaling할 때 처음 나타났으며(scaling laws), 그 결과 이러한 모델을 더욱 scaling하는 데 초점을 맞춘 연구가 진행되었다. (palm, Minerva)

부록, Scaling laws for neural language models

OpenAI 논문, Transformer기반 LM을 대상으로 경험적인 결과를 발견.

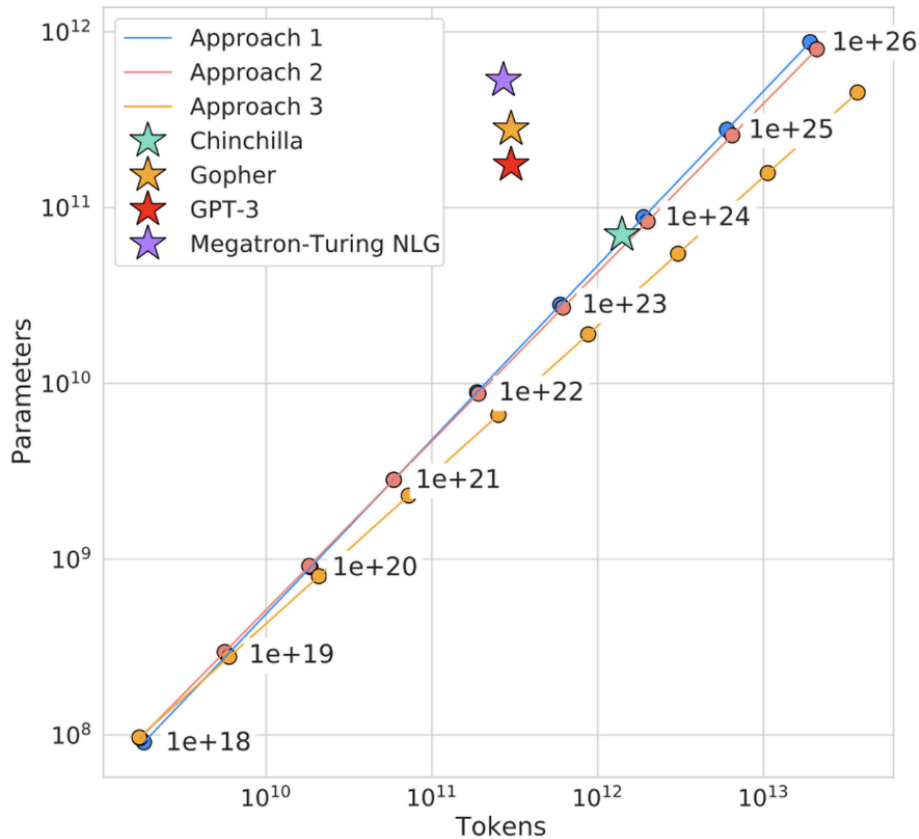
1. OpenAI의 연구에 따르면, Transformer 기반의 언어 모델 성능은 주로 모델 파라미터 수(N), 데이터 셋 크기(D), 그리고 필요한 컴퓨팅 양(C)에 의존함
2. 모델의 구조적 hyper-parameter (depth, width)의 영향은 상대적으로 매우 약함
3. 성능은 N, D, C 각각에 대해 power-law 관계를 보이며, 어떤 scale factor도 bottle-neck을 일으키지 않는 한 성능 개선이 예측 가능함
4. N과 D를 동시에 확장할 경우 성능이 개선되지만, N 또는 D중 하나만 증가시킬 경우 고정된 요소가 성능 패널티를 야기하여 성능 향상이 줄어듦. 예를 들어 N이 8배 증가하면 D도 약 5배 증가해야함(성능 패널티는 $(N^{0.74})/D$ 에 의존)
5. 학습 초기 곡선을 분석하면 장기간 학습을 통해 달성할 수 있는 loss를 예측할 수 있음
6. 큰 모델은 작은 모델보다 샘플 효율이 더 높아, 적은 최적화 단계와 더 적은 데이터로 같은 수준의 성능을 달성할 수 있음
7. 고정된 C 내에서 N과 D에 대한 제한이 없을 때, Very Large Model을 학습하면 가장 성능이 좋다. (추후 반박)

These efforts are based on the assumption that more parameters will lead to better performance.이 노력들은 많은 파라미터가 더 나은 성능을 이끌어낸다는 가정을 기반으로한다.

However, recent work from Hoffmann et al. (2022) shows that, for a given compute budget, the best performances are not achieved by the largest models, but by smaller models trained on more data.하지만 주어진 compute budget 에서 최고의 성능은 가장 큰 모델이 아니라 더 많은 데이터로 훈련된 더 작은 모델이었다.

부록: Training Compute-Optimal Large Language Models (Hoffmann et al.)

- Scaling laws을 반박(?)하는 Compute-Optimal Model 제안
- C가 고정되어있을 때, optimal 한 N과 D와의 관계가 존재함
 - (C: Compute budget, N: Num of params, D: Num of data(tokens))
- Scaling laws 결과와 달리 model performance를 위한 Model Size와 Training Tokens와의 관계는 거의 1:1의 weights를 가짐
 - C내에서 N과 D간의 관계를 찾기 위해 수 많은 실험을 진행 (약 400여개의 LM)
 - model size (params): 70M ~ 16B
 - tokens: 5B ~ 500B
- 이전에 DeepMind 에서 공개한 LLM Gopher 에 사용된 C로부터, Gopher의 optimal model은 현재보다 4배 더 작은 N과, 4배 더 많은 D를 사용해야 한다는 것을 예측함



The objective of the scaling laws from Hoffmann et al. (2022) is to determine how to best scale the dataset and model sizes for a particular training compute budget.

compute-optimal 논문에서 scaling laws의 목적은 정해진 compute budget에 맞게 데이터셋과 모델 크기를 잘 scale하는 법을 결정하는 것이다.

However, this objective disregards the inference budget, which becomes critical when serving a language model at scale.

하지만 이는 LLM을 제공할 때 중요해지는 inference budget을 무시한다.

In this context, given a target level of performance, the preferred model is not the fastest to train but the fastest at inference, and although it may be cheaper to train a large model to reach a certain level of performance, a smaller one trained longer will ultimately be cheaper at inference.

이런 문맥에서, 목표하는 성능이 주어졌을 때 선호하는 모델은 train 속도가 가장 빠른 모델이 아니라 inference 속도가 가장 빠른 모델이며, 특정 수준에 도달하기 위해 큰 모델을 훈련하는 것이 더 저렴할 수 있지만 궁극적으로는 더 오래 훈련된 작은 모델이 inference 비용이 더 저렴할 것이다.

For instance, although Hoffmann et al. (2022) recommends training a 10B model on 200B tokens, we find that the performance of a 7B model continues to improve even after 1T tokens.

예를 들어, 비록 호프만은 2000억 토큰과 100억짜리 모델을 추천하지만, 1T 토큰 이후에도 7B모델의 성능이 계속 향상되는 것을 확인했다.

The focus of this work is to train a series of language models that achieve the best possible performance at various inference budgets, by training on more tokens than what is typically used. 이 작업의 초점은 일반적으로 사용되는 것보다 더 많은 token으로 훈련하여 다양한 inference budgets에서 최상의 성능을 달성하는 일련의 LM을 훈련하는 것이다.

The resulting models, called LLaMA, ranges from 7B to 65B parameters with competitive performance compared to the LLaMA라고 하는 결과 모델은 70억에서 650억 개의 파라미터 범위에서 현존하는 최고의 LLM과 비교하여 경쟁력 있는 성능을 제공한다.

For instance, LLaMA-13B outperforms GPT-3 on most benchmarks, despite being 10× smaller.

예를 들어, LLaMA-13B는 10배 더 작음에도 불구하고 대부분의 벤치마크에서 GPT-3보다 성능이 뛰어나다.

We believe that this model will help democratize the access and study of LLMs, since it can be run on a single GPU.

이 모델은 단일 GPU에서 실행할 수 있기 때문에 LLM에 대한 접근과 학습을 대중화하는 데 도움이 될 것으로 믿는다.

At the higher-end of the scale, our 65B-parameter model is also competitive with the best large language models such as Chinchilla or PaLM-540B.

규모가 더 큰 경우, 65B-parameter 모델은 Chinchilla 또는 PaLM-540B와 같은 최고의 LLM과도 경쟁력이 있다.

Unlike Chinchilla, PaLM, or GPT-3, we only use publicly available data, making our work compatible with open-sourcing, while most existing models rely on data which is either not publicly available or undocumented (e.g. “Books – 2TB” or “Social media conversations”). There exist some exceptions, notably OPT (Zhang et al., 2022), GPT-NeoX (Black et al., 2022), BLOOM (Scao et al., 2022) and GLM (Zeng et al., 2022), but none that are competitive with PaLM-62B or Chinchilla.

대부분의 기존 모델이 공개적으로 사용 가능하지 않거나 문서화 되지 않은 데이터(예: "도서 - 2TB" 또는 "소셜 미디어 대화")에 의존하는 반면, 저자는 공개적으로 사용 가능한 데이터만 사용하므로 오픈 소싱과 호환된다. OPT, GPT-NeoX(Black 외, 2022), BLOOM, GLM 등 몇 가지 예외가 있지만 PaLM-62B나 Chinchilla와 경쟁할 만한 것은 없다. (기존에 비해 눈에 띄게 많이 사용하지 않았거나 다른 데이터들에 의해 묻혔다란 의미로 유추)

In the rest of this paper, we present an overview of the modifications we made to the transformer architecture (Vaswani et al., 2017), as well as our training method. We then report the performance of our models and compare with others LLMs on a set of standard benchmarks. Finally, we expose some of the biases and toxicity encoded in our models, using some of the most recent benchmarks from the responsible AI community.

이 논문의 나머지 부분에서는 transformer 아키텍처에 적용한 수정 사항(Vaswani 외, 2017)과 훈련 방법에 대한 개요를 소개한다. 그런 다음 모델의 성능을 보고하고 일련의 표준 벤치마크에서 다른 LLM과 비교한다. 마지막으로, 책임감 있는 AI 커뮤니티(hugging face로 유추)의 최신 벤치마크를 사용하여 모델에 인코딩된 biases와 toxicity(LLM에서 정치, 성희롱 등 부정적인 토큰들을 의미하는 듯) 중 일부를 노출한다.

Approach

LLaMA는 PaLM, GPT-3의 방법론에 따르고 chinchilla scaling laws를 따른다.

Pretraining-data

Dataset	Sampling prop.	Epochs	Disk size
CommonCrawl	67.0%	1.10	3.3 TB
C4	15.0%	1.06	783 GB
Github	4.5%	0.64	328 GB
Wikipedia	4.5%	2.45	83 GB
Books	4.5%	2.23	85 GB
ArXiv	2.5%	1.06	92 GB
StackExchange	2.0%	1.03	78 GB

위 표의 데이터셋을 혼합하여 제시된 비율만큼 사용했다. 이는 모두 공개적으로 접근 가능한 데이터셋이며, 위 데이터셋을 사용했을 때 1.4 조개의 토큰을 사용한다. 그리고 아래는 논문에서 제시된 데이터셋(tokenizer포함)에 대한 설명이다.

1. CommonCrawl Dataset

- 2017년부터 2020년까지 5개의 dump를 CCNet pipeline으로 처리
- 한 행 단위에서 중복을 제거하고 fastText 선형 분류기로 영어가 아닌 페이지 제거
- ngram으로 저품질 콘텐츠 필터링, 선형 모델로 reference로 분류되지 않은 페이지 폐기

2. C4 Dataset

- Raffle et al. 이 2020년에 제시한 데이터셋
- C4의 전처리에는 중복 제거 및 언어 식별 단계도 포함
- CCNet pipeline과의 주요 차이점은 구두점 유무나 웹페이지의 단어 및 문장 수와 같은 휴리스틱에 의존하는 filtering 을 사용

3. Github Dataset

- Google BigQuery에서 제공되는 공개 GitHub 데이터셋.
- 줄 길이, 영어/숫자 비율을 기반으로 한 휴리스틱으로 품질이 낮은 파일을 필터링
- 정규식으로 상용구 제거

4. Wikipedia Dataset

- 2022년 6월부터 8월까지 라틴어 또는 키릴 문자를 사용하는 20개 언어의 덤프를 추가
- 하이퍼링크를 제거하기 위한 데이터 처리

5. Gutenberg and Books3 Dataset

- 책을 포함하는 Gutenberg 프로젝트와 LLM 학습을 위한 public 데이터셋인 ThePile의 Book3 섹션의 두 가지 책 코퍼스
- 책 단위로 중복 제거를 수행해 90% 중복 제거

6. ArXiv Dataset

- 과학적 데이터를 추가하기 위해 LaTeX 파일 처리
- Lewkowycz 논문에 따라 section 앞 부분과 reference 모두 제거
- tex 파일에서 주석과 사용자가 작성한 inline 확장 및 macro를 제거하여 논문 간 일관성 확보

7. Stack Exchange Dataset

- 컴퓨터 과학, 화학 분야의 Q&A가 있는 Stack Exchange(Stack overflow와 유사한 커뮤니티)의 dump 사용
- HTML 태그를 제거한 후, 답변의 점수별로 정렬

8. Tokenizer

- BPE(Byte Pair Encoding) 사용
 - 기존에 있던 단어를 분리한 후 글자 단위에서 점차적으로 set을 만들어내는 Bottom Up 방식
 - 훈련 데이터에 있는 단어들은 모든 글자 혹은 유니코드 단위로 set을 만들고, 가장 많이 등장하는 unigram을 하나의 unigram으로 통합하는 방식
- SentencePiece의 구현을 사용.
- 모든 숫자를 개별 숫자로 분할 (4123 -> 4, 1, 2, 3)
- 알 수 없는 UTF-8 문자를 분해하기 위해 byte로 fallback.

Architecture

llama는 GPT 계열 모델과 유사한 구조로, 다음과 같은 주요 요소들로 구성된다:

- 토큰 임베딩 및 위치 인코딩:** 입력 토큰을 벡터로 변환하고, 토큰 간의 순서 정보를 추가하기 위해 위치 인코딩(예: 로터리 위치 임베딩)이 사용된다.
- 트랜스포머 블록:** 여러 개의 트랜스포머 블록이 순차적으로 쌓여 있으며, 각 블록은 주로 두 부분으로 나뉜다.
 - 다중 헤드 자기-어텐션:** 입력 시퀀스 내의 토큰들 간의 관계를 파악하기 위해 자기-어텐션 메커니즘이 사용된다.
 - 피드포워드 네트워크 (FFN):** 비선형 변환을 통해 정보를 가공하며, 각 블록마다 적용된다.
- 정규화 및 잔차 연결:** 안정적인 학습을 위해 RMSNorm과 같은 정규화 기법과, 잔차 연결(residual connection)이 적용된다.
- 최종 출력층:** 모델의 마지막에는 다음 토큰의 확률 분포를 계산하기 위한 linear layer다.

이처럼 llama는 Transformer 기반으로 구현되어 있으며 PaLM, GPT3, GPTNeo 방법론의 일부를 사용했다.

- Pre-normalization [GPT3]**
 - RMSNorm을 사용하여 훈련 안정성을 높이기 위해 하위 계층의 input을 정규화 하는 대신 output을 정규화한다
- SwiGLU activation function [PaLM]**
 - ReLU activation function을 SwiGLU로 대체.
 - $\text{Swish}(x) = x \cdot \sigma(x)$
 - $\text{GLU}(x, v) = x \cdot \sigma(v)$
 - $\text{SwiGLU}(x, v) = \text{Swish}(x) \cdot \sigma(v)$

- We use dimension of $(2/3)4d$ instead of $4d$ as a PaLM
- 주로 NLP에서 사용되며 Transformer모델의 효율성과 성능을 개선하는데 도움이 되는 activation function이다.
- **Rotary Embedding [GPTNeo]**
 - absolute positional embeddings 대신에 rotary positional embeddings(RoPE) 사용

params	dimension	n heads	n layers	learning rate	batch size	n tokens
6.7B	4096	32	32	$3.0e^{-4}$	4M	1.0T
13.0B	5120	40	40	$3.0e^{-4}$	4M	1.0T
32.5B	6656	52	60	$1.5e^{-4}$	4M	1.4T
65.2B	8192	64	80	$1.5e^{-4}$	4M	1.4T

Table 2: Model sizes, architectures, and optimization hyper-parameters.

- AdamW optimizer 사용 ($\beta_1 = 0.9$, $\beta_2 = 0.95$)
- cosine learning rate scheduler 사용
- weight decay 0.1 사용
- gradient clipping of 1.0 사용
- 2,000step warmup
- 모델 크기에 따른 Batch-size 및 lr 변경 (Table 2 참고)
- Training Speed를 개선하기 위해 몇 가지 optimization 수행했다.
- 메모리 사용량과 런타임을 줄이기 위해 multi-head attention의 효율적인 구현을 사용했다.
- xformers 라이브러리에서 사용할 수 있는 해당 최적화는 <Self-attention does not need $O(n^2)$ memory> 논문을 사용하였으며, Flashattention backward기법을 사용했다.
- 이는 모델 가중치를 저장하지 않고, Key/Query score를 계산하지 않음으로써 달성했다.
- 또한 training efficiency를 달성하기 위해, checkpointing으로 backward pass를 수행하는 동안 activations 수를 줄였다.
 - 더 정확하게는 Linear Layer의 output과 같이 compute 비용에 많이 드는 activation을 저장
 - 이는 PyTorch의 autograd에 의존하지 않고, transformer layers에 대한 backward function을 수동으로 구현하여 달성하였다.
- 해당 최적화 이점을 충분히 누리기 위해 <Reducing activation recomputation in large transformer models>에서 설명한 대로 model 및 sequence 병렬처리를 사용하여 모델의 memory usage를 줄여야한다.
- 또한 activation 연산과 네트워크를 통한 GPU간 통신(all_reduce연산으로 인해)도 가능한 한 겹치지 않도록 한다.
- 650억개 매개변수 모델을 훈련할 때, 2048개의 A100 GPU(80GB of RAM)에서 초당 380개의 토큰/GPU를 처리했다.
 - GPU만 약 563억원 어치다;
- 즉, 1.4T 개의 토큰에 대한 학습에 약 21일이 소요되었다.

Main Result

LLaMA는 GPT-3에 따라, zero-shot / few-shot learning을 고려하고 총 20개의 Benchmark에 대한 결과를 제시했다.

이때, 공개되지 않은 모델들인 GPT-3, Gopher, Chinchilla, PaLM, GPT-J, GPTNeo등 과 비교하고 Section 4에서는 LLaMA를 OPT-IML 및 FlanPaLM과 같은 명령어 조정 모델과 간략한 비교도 수행한다.

LLaMA는 free-form generation task 및 multiple choice task를 통해 평가하며 multiple choice task에서는 주어진 context를 기반으로 주어진 option 중에서 가장 적합한 completion을 선택하는 것이다.
즉, 가장 likelihood가 높은 option을 선택한다.

LLaMA는 대부분의 데이터셋에서 completion의 likelihood를 문자 수로 normalize하여 사용했다.

하지만 일부 데이터셋(OpenBookQA, BoolQ)에서는 completion의 likelihood를 "Answer:" context에서의 likelihood로 정규화하여 선택한다: $P(\text{completion}|\text{context}) / P(\text{completion} | \text{"Answer:"})$

3.7 Evolution of performance during training

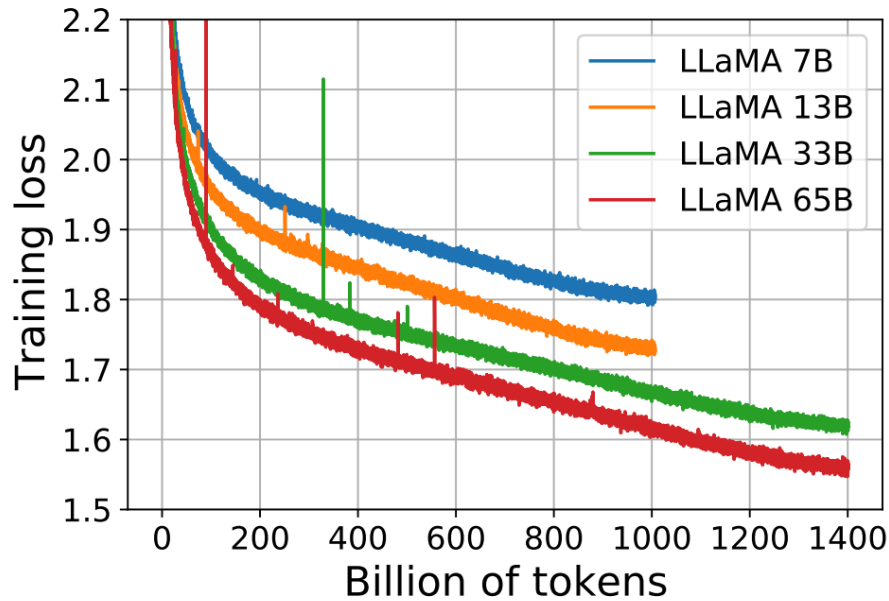


Figure 1: **Training loss over train tokens for the 7B, 13B, 33B, and 65 models.** LLaMA-33B and LLaMA-65B were trained on 1.4T tokens. The smaller models were trained on 1.0T tokens. All models are trained with a batch size of 4M tokens.

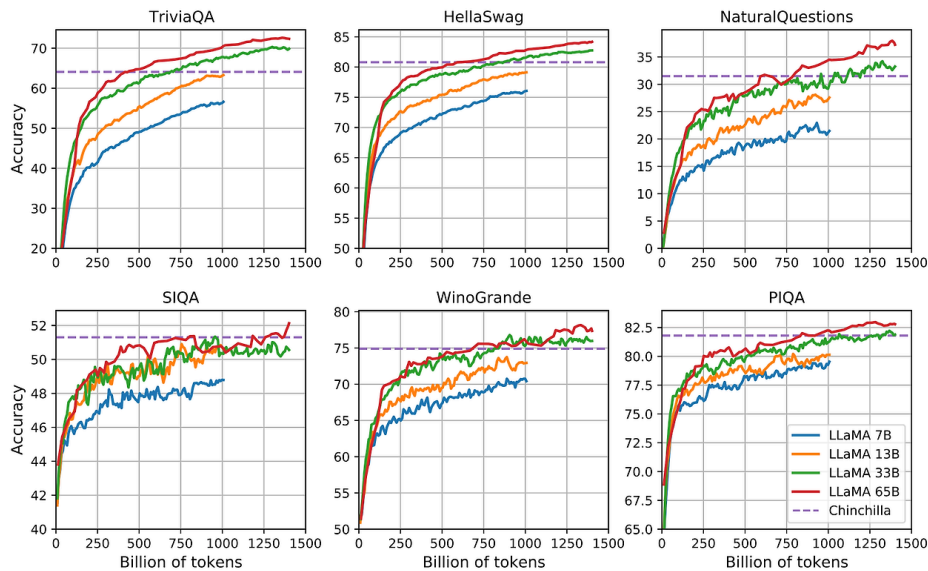


Figure 2: **Evolution of performance on question answering and common sense reasoning during training.**

Figure 2를 참고할 때, 대부분의 벤치마크에서 모델의 성능은 꾸준히 향상되고, 모델의 성능은 모델의 파라미터 수 즉 복잡성과 상관관계가 있는 것을 확인할 수 있다. (Figure 1 참고)

그러나 SIQA와 WinoGrande 벤치마크에서 예외사항을 볼 수 있는데, 이는 해당 벤치마크가 신뢰성이 떨어지는 것을 시사한다.

WinoGrande에서는 성능이 훈련 복잡도와 잘 상관되지 않으며 33B, 65B 모델이 훈련 중 유사한 성능을 보인다.

Instruction FineTuning

해당 섹션에서는 Instruction데이터를 간단히 fine-tuning하면 MMLU(Massive Multitask Language Understanding)성능이 빠르게 개선된다는 것을 보여준다.

Fine-tuning하지 않은 LLaMA-65B 버전도 이미 기본 Instruction를 따라갈 수 있지만, 아주 적은 양의 미세조정을 통해 MMLU의 성능이 향상되고 모델의 Instruction 수행 능력이 더욱 향상되는 것을 관찰할 수 있다.

OPT	30B	26.1
GLM	120B	44.8
PaLM	62B	55.1
PaLM-cont	62B	62.8
Chinchilla	70B	67.5
LLaMA	65B	63.4
OPT-IML-Max	30B	43.2
Flan-T5-XXL	11B	55.1
Flan-PaLM	62B	59.6
Flan-PaLM-cont	62B	66.1
LLaMA-I	65B	68.9

Table 10: Instruction finetuning – MMLU (5-shot).
Comparison of models of moderate size with and without instruction finetuning on MMLU.

LLaMA-I의 결과를 중간 크기의 기존 Instruction fine-tuning model 즉, OPT-IML과 Flan-PaLM시리즈와 비교하였다. 보고된 모든 수치는 해당 논문에서 가져왔으며 단순히 fine-tuning 했음에도 MMLU에서 68.9의 score를 도달해 모든 비슷한 param개수의 모델 보다 우수한 성능을 보였다.

요약

그동안의 Large Language Models(LLMs)들은 주로 Transformer를 backbone으로 하여 대규모 데이터셋을 대규모 컴퓨팅 리소스로 학습하여 그 성능을 경쟁하였습니다. 실상 그만큼의 budget이 없다면 이 시장에 뛰어들 수가 없겠죠. 학습 뿐만 아니라 추론 시에도 마찬가지입니다. 모델을 서비스하는 입장에서 생각해봅시다. 모델의 학습 성능은 좋은데, 애플 서비스하기에는 budget이 부족한 상황에 직면하게 됩니다. 학습 비용보다 추론 비용이 더 큰 상황이 됩니다. 역시나 현실적으로 모델을 채택하기엔 어렵겠죠. 이런 문제로 인해 compute budget을 고려한 연구가 진행되고 있는데요, LLaMA는 특히 추론 시 budget을 고려한 Foundation 모델입니다.

그러니까 결국은

효율적인 스케일링이다.

Llama-1은 7B부터 65B까지 다양한 파라미터 크기를 가진 모델들을 제시하면서, 소형 모델도 적절한 학습 기법과 데이터 전처리를 통해 기존의 대형 모델과 유사한 성능을 낼 수 있음을 보여준다. 그리고 공개된 데이터만 쓰고 open했다... 이점에서 뭔가 LM을 대중화 시키고 싶어하는 거 같았다.

관련 지식

fine tuning이란 pretrained된 언어 모델을 다운스트림 태스크에 맞춰 supervised 방식으로 데이터셋을 학습시키는 튜닝방법입니다. 튜닝이라는 단어에서도 보이듯이, pretrain 단계에서의 가중치가 task specific한 추가 데이터셋을 학습하면서 업데이트됩니다.

반면 **prompt learning**은 task-specific한 데이터를 추가로 학습시키지 않습니다. 사전학습을 진행할 때 모델에게 prompt(문제에 대한 설명)를 제공하여 학습시키는 방법입니다. 따라서 pretrained 모델의 추가적인 가중치 업데이트가 이뤄지지 않습니다.

prompt learning은 모델에게 예시를 몇 개를 주느냐에 따라서 zero-shot, one-shot, few-shot learning으로 구분됩니다. zero-shot은 모델에게 예시를 주지 않습니다. one-shot은 1개의 예시만을 제공하는 방법이고 few-shot은 몇 개의 예시를 보도록 허용하는 방법입니다.

니다.

Instruction tuning은 prompt learning의 prompt 방식과 finetuning의 가중치 업데이트를 결합한 튜닝 방법입니다. 모델에게 지시사항(instructions)으로 기술된 태스크를 supervised 방식으로 가르치는 것인데요, 이렇게 가르친다면 **모델이 지시사항을 따르는 방식을 학습하게 될 것이고, 따라서 unseen task를 마주했을 때도 지시사항을 따르는 방식을 이미 배웠기 때문에 unseen task의 지시사항에 따라 잘 추론할 수 있을 것이라는 아이디어**에서 착안하였습니다. 모티베이션은 모델을 지시사항(instructions)에 응답하도록 함으로써 모델의 제로샷 성능을 향상시키는 것입니다.